

CNN Basics and Why We Use Them

- Preserving Spatial Information: "Flattening" an image (turning a 2D grid of pixels into a 1D line for a standard dense neural network) destroys the spatial relationships between pixels. This is exactly why Convolutional Layers are used, they slide filters over the 2D image to extract features while maintaining spatial context.
- Network Depth Dynamics: As you go deeper into a CNN:
 - The number of filters (channels) increases to capture more complex features.
 - The spatial size (dimensions) of the image decreases due to pooling operations.
- Classification vs. Segmentation:* Classification requires discrete output neurons (e.g., "Is this a cat or dog?").
 - Segmentation requires the output to be an actual image specifically, a mask that selects or highlights specific regions at the pixel level.

The U-Net Architecture

- **Encoder (Downsampling):** The left side of the "U". It extracts features using 3 x 3 convolutions and reduces the spatial dimensions in half (1/2) using 2 x 2 Max Pooling. Here, the depth (channels) increases while the image size shrinks.
- **Bottleneck:** The lowest point of the network that bridges the encoder and decoder.
- **Decoder (Upsampling):** The right side of the "U". It rebuilds the image back to its original size through learning.

- It uses Up-convolution (specifically ConvTranspose2D) to scale the dimensions back up.
- **Skip Connections:** This is the most crucial part of U-Net. Because downsampling loses fine-grained location details, the network takes the output from an encoder layer and concatenates it in parallel with the corresponding decoder layer. This gives the decoder the "context" it needs to accurately place the mask over the original image.

Layer Mechanics and Training Rules

- **Padding:***
 - padding = "same": Zeros are added to the edges so the input and output sizes remain identical after a convolution.
 - padding = "valid": No padding is used, meaning the spatial dimensions will shrink slightly.
- **Batch Normalization:** Because gradients and activation values vary wildly during training, Batch Normalization normalizes the outputs of a layer using its mean and variance to stabilize the learning process.
- **Golden Rule of Testing:** Cannot update using the test dataset. During evaluation, you must use the rolling mean/variance calculated from the Train dataset.

Evaluation Metrics & Loss Functions

This section deals with how you measure if your segmentation mask is actually accurate.

- **IoU (Intersection over Union):***
 - Formula: $\text{IoU} = \text{Area of Overlap} / \text{Area of Union}$

- Larger is better. An IoU of 1 is perfect, 0 is a total failure.
- The Problem: IoU is not mathematically differentiable, meaning you cannot easily use it as a loss function for backpropagation.
- Dice Coefficient and Dice Loss:
 - Formula: $\text{Dice} = (2 \times \text{Area of Overlap}) / \text{Total Area}$
 - Because the Dice metric is differentiable, you can turn it into a loss function simply by calculating: $\text{Dice Loss} = 1 - \text{Dice}$.
 - Epsilon (ϵ): A tiny number added to the denominator to gracefully avoid a "divide by zero" error when there is no overlap.
- Combined Loss Function:
 - custom loss function: $\text{Loss} = 0.2 * \text{BCE} + 0.8 * \text{Dice}$.
 - BCE is Binary Cross-Entropy.
 - Your notes explain that more weight (0.8) is given to the Dice loss because it forces the network to focus heavily on getting the overlapping shape of the segmentation mask correct.
- Learning Rates: A brief note indicates that segmentation tasks usually require a lower learning rate (e.g., 0.05) compared to classification tasks.

The Mechanics of ConvTranspose2D(Up-Convolution)

A ConvTranspose2D layer takes a small image and turns it into a bigger image, while learning how to fill in the extra pixels.

Imagine you have a tiny pixel grid:

$$\begin{bmatrix} A & B \\ C & D \end{bmatrix}$$

A normal convolution would usually make this smaller.

But ConvTranspose2D does the opposite, it expands it.

How does it expand?

1. Each pixel “spreads out”

Instead of one pixel affecting one place, each input pixel affects multiple nearby pixels in the output.

2. These effects overlap

When multiple pixels spread out, their contributions overlap and add together.

3. The layer learns how to spread

It uses trainable weights, so it learns:

- where to put details
- how strong each contribution should be

Example with Stride = 1, Padding = 0

Example: ConvTranspose2D (2×2 → 3×3)

Input (low-resolution feature map)

$$X = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$$

Filter (kernel)

$$K = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

Settings

- stride = 1
- no padding

Contribution from X(0,0) = 1

Multiply kernel by 1:

$$\begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

Place it at top-left:

$$\begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{bmatrix}$$

2 Contribution from $X(0,1) = 2$

Multiply kernel by 2:

$$\begin{bmatrix} 2 & 0 \\ 0 & 2 \end{bmatrix}$$

Shift right by 1:

$$\begin{bmatrix} 0 & 2 & 0 \\ 0 & 0 & 2 \\ 0 & 0 & 0 \end{bmatrix}$$

3 Contribution from $X(1,0) = 3$

Multiply kernel by 3:

$$\begin{bmatrix} 3 & 0 \\ 0 & 3 \end{bmatrix}$$

Shift down by 1:

$$\begin{bmatrix} 0 & 0 & 0 \\ 3 & 0 & 0 \\ 0 & 3 & 0 \end{bmatrix}$$

4 Contribution from $X(1,1) = 4$

Multiply kernel by 4:

$$\begin{bmatrix} 4 & 0 \\ 0 & 4 \end{bmatrix}$$

Shift down-right:

$$\begin{bmatrix} 0 & 0 & 0 \\ 0 & 4 & 0 \\ 0 & 0 & 4 \end{bmatrix}$$

+ Final Output (Sum Everything)

Add all contributions:

$$Y = \begin{bmatrix} 1 & 2 & 0 \\ 3 & (1+4) & 2 \\ 0 & 3 & 4 \end{bmatrix} = \begin{bmatrix} 1 & 2 & 0 \\ 3 & 5 & 2 \\ 0 & 3 & 4 \end{bmatrix}$$

Example with Stride = 2, Padding = 0

Given

Input

$$X = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$$

Kernel

$$K = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

Settings

- stride = 2
- padding = 0

When doing ConvTranspose2D, the correct output size is:

$$(2 - 1) \times 2 + 2 = 4$$

So the output should be 4×4 , not 2×2 .

1 From $X(0,0) = 1$

Place at (0,0):

$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix}$$

2 From $X(0,1) = 2$ (shift right by 2)

$$\begin{bmatrix} 0 & 0 & 2 & 0 \\ 0 & 0 & 0 & 2 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix}$$

3 From $X(1,0) = 3$ (shift down by 2)

$$\begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 3 & 0 & 0 & 0 \\ 0 & 3 & 0 & 0 \end{bmatrix}$$

4 From $X(1,1) = 4$ (shift down-right by 2)

$$\begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 4 & 0 \\ 0 & 0 & 0 & 4 \end{bmatrix}$$

+ Final Output (Add all)

$$Y = \begin{bmatrix} 1 & 0 & 2 & 0 \\ 0 & 1 & 0 & 2 \\ 3 & 0 & 4 & 0 \\ 0 & 3 & 0 & 4 \end{bmatrix}$$

Example with Stride = 2, Padding = 1

ConvTranspose is basically: “reverse geometry of convolution”

In normal convolution:

- padding → adds zeros → increases output

In transposed convolution:

- padding → removes borders → decreases output

For ConvTranspose2D, output size is:

$$\text{Output} = (N - 1) \cdot s - 2p + k$$

Where:

- N = input size
- s = stride
- p = padding
- k = kernel size

👉 Padding reduces the output size (this is the unintuitive part)

Input (2×2)

$$X = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$$

Kernel (2×2)

$$K = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

Case 1: No padding (p = 0)

$$(2 - 1) \cdot 2 + 2 = 4$$

Output = 4×4 (what we already computed):

$$\begin{bmatrix} 1 & 0 & 2 & 0 \\ 0 & 1 & 0 & 2 \\ 3 & 0 & 4 & 0 \\ 0 & 3 & 0 & 4 \end{bmatrix}$$

Case 2: Padding = 1

Now apply formula:

$$(2 - 1) \cdot 2 - 2(1) + 2 = 2$$

👉 Output becomes 2×2

What actually happens?

Padding in ConvTranspose2D means:

👉 you crop the borders of the full output

So instead of recomputing everything, we:

1. Compute the full 4×4 output (same as before)
2. Remove 1 layer from all sides

Cropping step

Original:

$$\begin{bmatrix} 1 & 0 & 2 & 0 \\ 0 & 1 & 0 & 2 \\ 3 & 0 & 4 & 0 \\ 0 & 3 & 0 & 4 \end{bmatrix}$$

Remove 1 pixel border:

$$Y = \begin{bmatrix} 1 & 0 \\ 0 & 4 \end{bmatrix}$$

Given:

An input feature map I and a convolutional kernel (filter) K defined as follows:

- Input Image (I):

$$\begin{bmatrix} 5 & 4 \\ 3 & 2 \end{bmatrix}$$

- Filter (K):

$$\begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix}$$

Task:

Perform an **up-convolution** (transposed convolution) operation using a **stride of 1** and **no padding**.

Solution:

Each pixel in the 2 x 2 input is multiplied by the entire 2 x 2 filter. The results are placed in the output grid starting at the position corresponding to the input pixel, and overlapping values are summed.

1. Input (0, 0) = 5

$$5 \times \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix} = \begin{bmatrix} 10 & 5 & . \\ 5 & 10 & . \\ . & . & . \end{bmatrix}$$

2. Input (0, 1) = 4 (Shifted right by 1)

$$4 \times \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix} = \begin{bmatrix} . & 8 & 4 \\ . & 4 & 8 \\ . & . & . \end{bmatrix}$$

3. Input (1, 0) = 3 (Shifted down by 1)

$$3 \times \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix} = \begin{bmatrix} . & . & . \\ 6 & 3 & . \\ 3 & 6 & . \end{bmatrix}$$

4. Input (1, 1) = 2 (Shifted right 1, down 1)

$$2 \times \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix} = \begin{bmatrix} . & . & . \\ . & 4 & 2 \\ . & 2 & 4 \end{bmatrix}$$

Final Summation

Summing all the overlapping matrices:

- Top row: 10, (5 + 8), 4
- Middle row: (5 + 6), (10 + 4 + 3 + 4), (8 + 2)
- Bottom row: 3, (6 + 2), 4

Resulting Matrix:

$$\begin{bmatrix} 10 & 13 & 4 \\ 11 & 21 & 10 \\ 3 & 8 & 4 \end{bmatrix}$$

Skip Connections in U-Net

Skip connections pass feature maps from the encoder directly to the decoder and concatenate them with upsampled features.

Why needed:

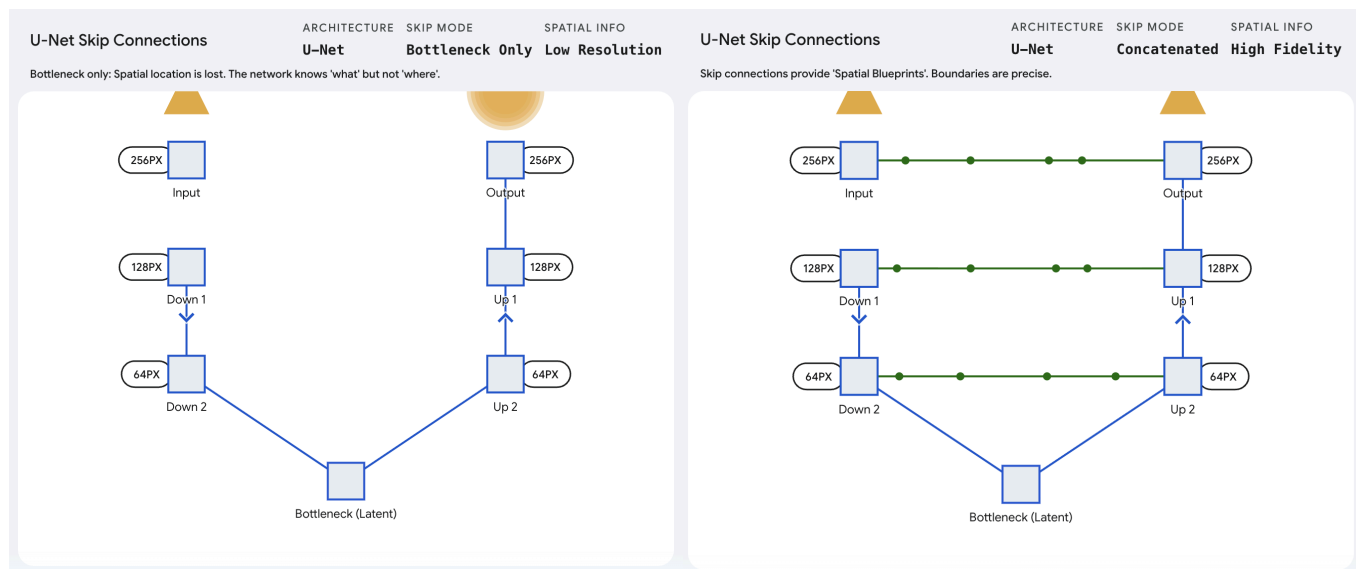
During encoding (convolution + pooling), the model learns what is in the image but loses precise where (spatial details) due to downsampling.

What they do:

They provide the decoder with high-resolution spatial information from early layers, helping it reconstruct accurate boundaries instead of blurry outputs.

In short:

- Encoder → semantic context (“what”)
- Skip connections → spatial details (“where”)
- Decoder combines both → precise segmentation mask



Loss Calculation

Intersection Over Union(IoU)

$$IoU = \frac{\text{Area of Overlap}}{\text{Area of Union}} = \frac{|A \cap B|}{|A \cup B|} = \frac{TP}{TP + FP + FN}$$

Mean Intersection Over Union(mIoU)

$$mIoU = \frac{1}{C} \sum_{i=1}^C IoU_i$$

Dice Loss

Formula (Coefficient):

$$DSC = \frac{2|A \cap B|}{|A| + |B|} = \frac{2 \cdot P(A) \cdot P(B)}{P(A) + P(B)}$$

Formula (Loss):

$$L_{\text{Dice}} = 1 - DSC$$

Example 1

Ground Truth (Y)

$$\begin{bmatrix} 1 & 1 & 0 \\ 1 & 1 & 0 \\ 0 & 0 & 0 \end{bmatrix}$$

(Total Area = 4 pixels)

Model Prediction ($\hat{Y}$)

$$\begin{bmatrix} 0 & 1 & 1 \\ 0 & 1 & 1 \\ 0 & 0 & 0 \end{bmatrix}$$

(Total Area = 4 pixels)

Find IoU and Dice Loss

IoU

Step 1: Count pixels

Ground Truth Y

$$\begin{bmatrix} 1 & 1 & 0 \\ 1 & 1 & 0 \\ 0 & 0 & 0 \end{bmatrix}$$

Number of 1s:

- $P(A) = 4$
-

Prediction $\hat{Y}$

$$\begin{bmatrix} 0 & 1 & 1 \\ 0 & 1 & 1 \\ 0 & 0 & 0 \end{bmatrix}$$

Number of 1s:

- $P(B) = 4$

Step 2: Compute intersection (important!)

We need overlap (same position = 1 in both):

$$Y \cap \hat{Y} = \begin{bmatrix} 0 & 1 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{bmatrix}$$

Number of overlapping 1s:

- $= 2$

So:

- $|A \cap B| = 2$

Step 3: Compute union

$$\begin{aligned}|A \cup B| &= |A| + |B| - |A \cap B| \\ &= 4 + 4 - 2 = 6\end{aligned}$$

Step 4: IoU

$$\text{IoU} = \frac{|A \cap B|}{|A \cup B|}$$

$$\text{IoU} = \frac{2}{6} = \frac{1}{3}$$

Dice Loss

Step 1: Count pixels

Ground Truth Y

$$\begin{bmatrix} 1 & 1 & 0 \\ 1 & 1 & 0 \\ 0 & 0 & 0 \end{bmatrix}$$

Number of 1s:

- $P(A) = 4$

Prediction $\hat{Y}$

$$\begin{bmatrix} 0 & 1 & 1 \\ 0 & 1 & 1 \\ 0 & 0 & 0 \end{bmatrix}$$

Number of 1s:

- $P(B) = 4$

Step 2: Compute intersection (important!)

We need overlap (same position = 1 in both):

$$Y \cap \hat{Y} = \begin{bmatrix} 0 & 1 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{bmatrix}$$

Number of overlapping 1s:

- = 2

So:

- $|A \cap B| = 2$

Step 3: Dice coefficient

$$\begin{aligned} \text{Dice} &= \frac{2 \cdot 2}{4 + 4} \\ &= \frac{4}{8} = 0.5 \end{aligned}$$

So, Dice Loss = 1 - Dice Coefficient = 1 - 0.5 = 0.5

Example 2

Ground Truth

1	1	0	0	0
1	1	0	0	0
0	0	1	1	0
0	0	1	1	0
0	0	0	0	0

Predicted Mask

1	0	0	0	0
1	1	0	0	0
0	0	1	0	0
0	0	1	1	1
0	0	0	0	0

Find IoU and Dice Loss.

From Ground Truth, $P(A) = 8$

From Predicted, $P(B) = 7$

$$|A \cap B| = 6$$

$$|A \cup B| = |A| + |B| - |A \cap B| = 8 + 7 - 6 = 9$$

$$\text{IoU} = |A \cap B| / |A \cup B| = 6/9 = 0.667$$

$$\text{Dice Loss} = 1 - (2 * |A \cap B|) / (P(A) + P(B)) = 1 - (2 * 6) / (8 + 7) = 0.2$$

Another Approach

TP = 6, FP = 1, FN = 2, TN = 16

$$\text{IoU} = \text{TP} / (\text{TP} + \text{FP} + \text{FN}) = 6 / (6 + 1 + 2) = 0.667$$

$$\text{Dice Coefficient} = 2\text{TP} / (2\text{TP} + \text{FP} + \text{FN}) = (2 * 6) / ((2 * 6) + 1 + 2) = 0.8$$

$$\text{Dice Loss} = 1 - \text{Dice Coefficient} = 1 - 0.8 = 0.2$$

True Positives (TP): Where both are 1.

False Positives (FP): Where Prediction is 1 but Truth is 0.

False Negatives (FN): Where Prediction is 0 but Truth is 1.

True Negatives (TN): Where both are 0.